

Đáp Án Bài Tập Cuối Chương

`src/App.jsx` (Áp dụng `React.lazy` và `Suspense`)

```
import React, { Suspense, lazy } from 'react'; // Import Suspense và lazy
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import NavBar from './components/NavBar/NavBar';
// Import trực tiếp ban đầu - Sẽ đổi thành React.lazy ở Bước 1
// import LandingPage from './screens/LandingPage/LandingPage';
// import ProductsPage from './screens/ProductsPage/ProductsPage';

// Sử dụng React.lazy để import động các component màn hình
const LandingPage = lazy(() => import('./screens/LandingPage/LandingPage'));
const ProductsPage = lazy(() => import('./screens/ProductsPage/ProductsPage'));

function App() {
  return (
    <BrowserRouter>
      <NavBar />
      {/* Bọc Routes bằng Suspense và cung cấp fallback UI */}
      <Suspense fallback={<div>Loading page content...</div>}>
        <Routes>
          <Route path="/" element={<LandingPage />} />
          <Route path="/products" element={<ProductsPage />} />
          {/* Route 404 đơn giản */}
          <Route path="*" element={<div style={{padding: '40px', textAlign: 'center'}}><h2>404 - Page Not Found</h2></div>} />
        </Routes>
      </Suspense>
    </BrowserRouter>
  );
}

export default App;
```

`src/components/NavBar/NavBar.jsx` (Áp dụng `React.memo`)

```
import React, { memo } from 'react'; // Import React
import { Link } from 'react-router-dom';
import './NavBar.css'; // Import CSS

// Sử dụng React.memo để bọc component
function NavBar() {
  console.log("Rendering NavBar");
  return (
    <nav className="NavBar">
      <Link to="/" className="NavBar-link">Home</Link>
      <Link to="/products" className="NavBar-link">Products</Link>
    </nav>
  );
}; // Kết thúc React.memo

export default memo(NavBar);
```

`src/components/ProductItem/ProductItem.jsx` (Áp dụng `React.memo`)

```
import React, { memo } from 'react'; // Import React
import './ProductItem.css'; // Import CSS

// Sử dụng React.memo để bọc component
function ProductItem({ product, onAddToCart }) {
  // Thêm console.log để theo dõi render
  console.log(`Rendering ProductItem: ${product.id} - ${product.name}`);
  return (
    <li className="ProductItem">
      <div className="ProductItem-details">
        <h4 className="ProductItem-name">{product.name}</h4>
        <p className="ProductItem-category">Category: {product.category}</p>
        <p className="ProductItem-price">Price: ${product.price}</p>
      </div>
      <button className="ProductItem-button" onClick={() => onAddToCart(product.id)}>
        Add to Cart
      </button>
    </li>
  );
}; // Kết thúc React.memo

export default memo(ProductItem);
```

src/components/CategoryFilter/CategoryFilter.jsx (Áp dụng React.memo)

```
import React, { memo } from 'react'; // Import React
import './CategoryFilter.css'; // Import CSS

// Sử dụng React.memo để bọc component
function CategoryFilter({ categories, currentCategory, onCategoryChange }) {
  console.log("Rendering CategoryFilter");
  return (
    <div className="CategoryFilter">
      <label htmlFor="category-select" className="CategoryFilter-label">Filter by Category:</label>
      <select
        id="category-select"
        className="CategoryFilter-select"
        value={currentCategory}
        onChange={(e) => onCategoryChange(e.target.value)}
      >
        {categories.map(category => (
          <option key={category} value={category}>{category}</option>
        ))}
      </select>
    </div>
  );
}; // Kết thúc React.memo

export default memo(CategoryFilter);
```

src/screens/ProductsPage/ProductsPage.jsx (Áp dụng useMemo và useCallback)

```
import React, { useState, useMemo, useCallback } from 'react'; // Import useCallback
import ProductList from '../components/ProductList/ProductList';
import CategoryFilter from '../components/CategoryFilter/CategoryFilter';
import { getProducts, getCategories } from '../data/products';
import './ProductsPage.css'; // Import CSS
```

```

function ProductsPage() {
  console.log("Rendering ProductsPage");
  const [allProducts, setAllProducts] = useState(() => getProducts());
  const [selectedCategory, setSelectedCategory] = useState('All Categories');
  const [cartItems, setCartItems] = useState([]);
  const [counter, setCounter] = useState(0); // State giả để test re-render

  // Categories được tính toán lại chỉ khi allProducts thay đổi
  const categories = useMemo(() => {
    console.log("Calculating categories...");
    return getCategories(allProducts);
  }, [allProducts]);

  // Logic lọc được tối ưu bằng useMemo
  const filteredProducts = useMemo(() => {
    console.log("Filtering logic running..."); // Log bên trong useMemo
    return allProducts.filter(product => {
      return selectedCategory === 'All Categories' || product.category === selectedCategory;
    });
  }, [allProducts, selectedCategory]); // Dependencies: chỉ chạy lại khi sản phẩm hoặc category thay đổi

  // Hàm xử lý category change được tối ưu bằng useCallback
  const handleCategoryChange = useCallback((newCategory) => {
    setSelectedCategory(newCategory);
  }, []); // Dependency rỗng vì setSelectedCategory là ổn định

  // Hàm xử lý add to cart được tối ưu bằng useCallback và functional update
  const handleAddToCart = useCallback((productId) => {
    console.log(`Adding product ${productId} to cart`);
    // Sử dụng functional update để không phụ thuộc vào `cartItems` trong dependencies
    setCartItems(prevItems => {
      if (!prevItems.includes(productId)) {
        return [...prevItems, productId];
      }
      console.log(`Product ${productId} already in cart.`);
      return prevItems; // Trả về state cũ nếu đã tồn tại
    });
  }, []); // Dependency rỗng

  const handleRefreshProducts = useCallback(() => {
    console.log("Refreshing product list...");
    // Nên tạo ra một ID mới thực sự hoặc thay đổi cấu trúc để đảm bảo re-render nếu cần
    // Cách làm này đảm bảo object reference thay đổi
    setAllProducts(getProducts().map(p => ({ ...p, _id: Math.random() })));
  }, []); // Dependency rỗng vì getProducts() không phụ thuộc state/props hiện tại

  return (
    <div className="ProductsPage">
      <h1>Product Catalog</h1>
      <div className="ProductsPage-controls">
        <button onClick={handleRefreshProducts}>Refresh Product List</button>
        <p className="ProductsPage-cartInfo">Items in Cart: {cartItems.length}</p>
        <button onClick={() => setCounter(c => c + 1)}>
          Force Re-render (Counter: {counter})
        </button>
      </div>
      <CategoryFilter
        categories={categories}
        currentCategory={selectedCategory}
      />
    </div>
  );
}

```

```
        onCategoryChange={handleCategoryChange} // Truyền callback đã ổn định
      />
      {/* ProductList nhận callback đã ổn định */}
      <ProductList products={filteredProducts} onAddToCart={handleAddToCart} />
    </div>
  );
}

export default ProductsPage;
```